```
Start coding or generate with AI.
```

Generative AI Text Generation Assignment

Student Name

Pius Nnaji

Project Overview

This project explores Generative AI and GPT-style text generation. A simple text generation model is trained using a public-domain text dataset from Project Gutenberg. The notebook explains GPT architecture, tokenization, attention, sequence generation, and ethical considerations.

```
# Import required libraries

import numpy as np
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers
import matplotlib.pyplot as plt

print("TensorFlow version:", tf.__version__)
```

```
TensorFlow version: 2.20.0
```

```
# Load a public-domain text from Project Gutenberg

import urllib.request

url = "https://www.gutenberg.org/files/11/11-0.txt"
filename = "alice_in_wonderland.txt"

urllib.request.urlretrieve(url, filename)

with open(filename, "r", encoding="utf-8") as file:
    text = file.read()

print("Total characters in dataset:", len(text))
print(text[:1000])
```

```
Total characters in dataset: 144696
*** START OF THE PROJECT GUTENBERG EBOOK 11 ***
```

```
[Illustration]




Alice’s Adventures in Wonderland

by Lewis Carroll

THE MILLENNIUM FULCRUM EDITION 3.0

Contents

 CHAPTER I.     Down the Rabbit-Hole
 CHAPTER II.    The Pool of Tears
 CHAPTER III.   A Caucus-Race and a Long Tale
 CHAPTER IV.    The Rabbit Sends in a Little Bill
 CHAPTER V.     Advice from a Caterpillar
 CHAPTER VI.    Pig and Pepper
 CHAPTER VII.   A Mad Tea-Party
 CHAPTER VIII.  The Queen’s Croquet-Ground
 CHAPTER IX.    The Mock Turtle’s Story
 CHAPTER X.     The Lobster Quadrille
 CHAPTER XI.    Who Stole the Tarts?
 CHAPTER XII.   Alice’s Evidence




CHAPTER I.
Down the Rabbit-Hole


Alice was beginning to get very tired of sitting by her sister on the
bank, and of having nothing to do: once or twice she had peeped into
the book her sister was reading, but it had no pictures or
conversations in it, “and what is the use of a book,” thought Alice
“without pictures or conversations?”

So she was considering in her
```

```
# Clean and reduce the text for faster training

text = text.lower()

# Use part of the text to keep training faster in Colab
text = text[:200000]

print("Cleaned text length:", len(text))
print(text[:500])
```

```
Cleaned text length: 144696
*** start of the project gutenberg ebook 11 ***
```

```
[illustration]




alice’s adventures in wonderland

by lewis carroll

the millennium fulcrum edition 3.0

contents

 chapter i.     down the rabbit-hole
 chapter ii.    the pool of tears
 chapter iii.   a caucus-race and a long tale
 chapter iv.    the rabbit sends in a little bill
 chapter v.     advice from a caterpillar
 chapter vi.    pig and pepper
 chapter vii.   a mad tea-party
 chapter viii.  the queen’s croquet-ground
 chapter ix.    the
```

Gemini

```
=# Create vocabulary of unique characte
# Create vocabulary of unique character

chars = sorted(list(set(text)))

char_to_index = {char: index for index,
index_to_char = {index: char for index,

print("Total unique characters:", len(
print("Characters:", chars)
```

```
  File "/tmp/ipykernel_11303/3805976806.py", line 1
    =# Create vocabulary of unique characters
    ^
SyntaxError: invalid syntax
```

Next steps: Explain error

```
# STEP 5: Create character vocabulary

chars = sorted(list(set(text)))

char_to_index = {}
index_to_char = {}

for index, char in enumerate(chars):
```

```
    char_to_index[char] = index
    index_to_char[index] = char

print("Total unique characters:", len(chars))
print("Characters:", chars)
```

```
Total unique characters: 50
Characters: ['\n', ' ', '!', '(', ')', '*', ',', '-', '.', '0', '1', '3', ':', '
```

Start coding or generate with AI.

Start coding or generate with AI.

Start coding or generate with AI.

```
char_to_index = {char: index for index, char in enumerate(chars)}
index_to_char = {index: char for index, char in enumerate(chars)}

encoded_text = np.array([char_to_index[char] for char in text])

print("Encoded text shape:", encoded_text.shape)
print("First 20 encoded characters:", encoded_text[:20])
```

```
Encoded text shape: (144696,)
First 20 encoded characters: [ 5  5  5  1 36 37 18 35 37  1 32 23  1 37 25 22  1
```

```
# STEP 7: Create input and target sequences

sequence_length = 100
step = 3

input_sequences = []
target_chars = []

for i in range(0, len(encoded_text) - sequence_length, step):
    input_sequences.append(encoded_text[i:i + sequence_length])
    target_chars.append(encoded_text[i + sequence_length])

X = np.array(input_sequences)
y = np.array(target_chars)

print("Input shape:", X.shape)
print("Target shape:", y.shape)
```

```
Input shape: (48199, 100)
Target shape: (48199,)
```

```
# STEP 8: Build a simple text generation model

vocab_size = len(chars)

model = keras.Sequential([
    layers.Embedding(input_dim=vocab_size, output_dim=64),
    layers.LSTM(128),
    layers.Dense(vocab_size, activation="softmax")
])

model.compile(
    optimizer="adam",
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"]
)

model.summary()
```

Model: "sequential"

Layer (type)	Output Shape	Param #
embedding (Embedding)	?	0 (unbuilt)
lstm (LSTM)	?	0 (unbuilt)
dense (Dense)	?	0 (unbuilt)

Total params: 0 (0.00 B)
Trainable params: 0 (0.00 B)
Non-trainable params: 0 (0.00 B)

```
# STEP 9: Train the text generation model

history = model.fit(
    X,
    y,
    batch_size=64,
    epochs=10,
    validation_split=0.2
)
```

```
Epoch 1/10
603/603 ━━━━━━━━━━━━━━━━━━━━ 138s 225ms/step - accuracy: 0.2741 - loss: 2.6349 -
Epoch 2/10
603/603 ━━━━━━━━━━━━━━━━━━━━ 148s 245ms/step - accuracy: 0.3638 - loss: 2.2259 -
Epoch 3/10
603/603 ━━━━━━━━━━━━━━━━━━━━ 0s 204ms/step - accuracy: 0.3938 - loss: 2.0992
```

```
# STEP 10: Plot training and validation accuracy
```

```
import matplotlib.pyplot as plt

plt.figure(figsize=(8, 5))
plt.plot(history.history["accuracy"], label="Training Accuracy")
plt.plot(history.history["val_accuracy"], label="Validation Accuracy")
plt.title("Training and Validation Accuracy")
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.legend()
plt.grid(True)
plt.show()
```

```
# STEP 11: Plot training and validation loss

plt.figure(figsize=(8, 5))
plt.plot(history.history["loss"], label="Training Loss")
plt.plot(history.history["val_loss"], label="Validation Loss")
plt.title("Training and Validation Loss")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.legend()
plt.grid(True)
plt.show()
```

```
# STEP 12: Function to generate new text from a seed phrase

import numpy as np

def generate_text(seed_text, length=500, temperature=0.8):
    generated = seed_text.lower()

    for _ in range(length):
        input_text = generated[-sequence_length:]

        # If the seed is shorter than the required sequence length, add spaces
        if len(input_text) < sequence_length:
            input_text = " " * (sequence_length - len(input_text)) + input_text

        input_encoded = np.array([char_to_index.get(char, 0) for char in input_
        input_encoded = input_encoded.reshape(1, sequence_length)

        predictions = model.predict(input_encoded, verbose=0)[0]

        # Adjust randomness using temperature
        predictions = np.log(predictions + 1e-8) / temperature
        predictions = np.exp(predictions) / np.sum(np.exp(predictions))

        predicted_index = np.random.choice(len(chars), p=predictions)
        predicted_char = index_to_char[predicted_index]
```

```
            generated += predicted_char

        return generated
```

```
# STEP 13: Generate text using a seed input

seed = "alice was beginning"
generated_output = generate_text(seed, length=500, temperature=0.8)

print(generated_output)
```

```
# STEP 14: Practical content creation example

content_seed = "once upon a time"
content_output = generate_text(content_seed, length=700, temperature=0.7)

print("Generated Creative Writing Example:")
print(content_output)
```

GPT Architecture Explanation

Generative Pre-trained Transformers, also known as GPTs, are advanced language models based on the transformer architecture. They are designed to generate human-like text by predicting the next token in a sequence.

Transformer Model

The transformer architecture uses layers of attention mechanisms instead of traditional recurrent neural networks. This allows the model to process text efficiently and understand relationships between words across long sequences.

Attention Mechanism

Attention helps the model decide which words or tokens are most important when predicting the next token. For example, in a sentence, the model can focus on earlier words that provide useful context for the next word.

Tokenization

Before training, text is broken into smaller units called tokens. A token can be a word, part of a word, punctuation mark, or character. GPT models use tokens as numerical inputs.

Probability and Sequence Generation

GPT models generate text by predicting the probability of the next token. The model selects a likely next token, adds it to the sequence, and repeats this process to create longer text.

Training Process

GPTs are trained on large amounts of text data. During training, the model learns language patterns by repeatedly predicting the next token. Over time, it improves its ability to generate coherent and context-aware text.

Close

```
# Final Report: Generative AI
Text Generation

## Student Name
Pius Nnaji

## Introduction
Generative Artificial
Intelligence is a branch of AI
that focuses on creating new
content such as text, images,
music, and code. Text
generation is one of the most
common applications of
Generative AI because it allows
machines to produce written
content based on a prompt or
seed input. This project
explores how a simple text
generation model can be trained
using a public-domain text
dataset.

## Dataset Description
The dataset used in this
project was Alice’s Adventures
in Wonderland from Project
Gutenberg. Project Gutenberg
provides access to
public-domain books that can be
used for educational and
research purposes. The text was
loaded into Google Colab,
converted to lowercase, and
```

Final Report: Generative AI Text Generation

Student Name

Pius Nnaji

Introduction

Generative Artificial Intelligence is a branch of AI that focuses on creating new content such as text, images, music, and code. Text generation is one of the most common applications of Generative AI because it allows machines to produce written content based on a prompt or seed input. This project explores how a simple text generation model can be trained using a public-domain text dataset.

Dataset Description

The dataset used in this project was Alice’s Adventures in Wonderland from Project Gutenberg. Project Gutenberg provides

```
prepared for model training.

## GPT Architecture and
Functionality
Generative Pre-trained
Transformers are based on the
transformer architecture. GPT
models use tokenization to
break text into smaller units
and use attention mechanisms to
understand relationships
between tokens. The model
predicts the next token based
on previous tokens, then
repeats this process to
generate a full sequence of
text.

Although this project uses a
simpler LSTM-based model
instead of a full GPT model,
the main idea is similar: the
model learns patterns from text
and generates new text by
predicting what should come
next.

## Methodology
The text was cleaned and
converted into numerical
format. A character-level
vocabulary was created, where
each unique character was
assigned a number. Training
sequences were created using
100 characters as input and the
next character as the target.

A neural network was built
using an Embedding layer, an
LSTM layer, and a Dense output
layer with Softmax activation.
The model was compiled using
the Adam optimizer and sparse
categorical crossentropy loss.
The model was trained for 10
epochs with a validation split
of 20%.

## Findings
The model was able to generate
new text based on a seed
```

access to public-domain books that can be used for educational and research purposes. The text was loaded into Google Colab, converted to lowercase, and prepared for model training.

GPT Architecture and Functionality

Generative Pre-trained Transformers are based on the transformer architecture. GPT models use tokenization to break text into smaller units and use attention mechanisms to understand relationships between tokens. The model predicts the next token based on previous tokens, then repeats this process to generate a full sequence of text.

Although this project uses a simpler LSTM-based model instead of a full GPT model, the main idea is similar: the model learns patterns from text and generates new text by predicting what should come next.

Methodology

The text was cleaned and converted into numerical format. A character-level vocabulary was created, where each unique character was assigned a number. Training sequences were created using 100 characters as input and the next character as the target.

A neural network was built using an Embedding layer, an LSTM layer, and a Dense output layer with Softmax activation. The model was compiled using the Adam optimizer and sparse categorical

```
phrase. The generated text
followed some patterns from the
training data, such as word
spacing, punctuation, and
sentence-like structure.
However, because the model was
small and trained for a limited
number of epochs, the generated
output was not always
grammatically perfect.

Training and validation
accuracy and loss graphs were
used to monitor the learning
process. These visualizations
helped show whether the model
improved during training.

## Application Demonstration
A practical content creation
example was implemented using a
seed phrase such as “once upon
a time.” The trained model
generated a short creative
writing sample. This
demonstrates how text
generation can support
applications such as
storytelling, writing
assistance, chatbot responses,
and content drafting.

## Ethical Considerations
Generative AI can create useful
content, but it also raises
ethical concerns. These include
misinformation, plagiarism,
biased text, and overreliance
on AI-generated material. To
reduce these risks, generated
content should be reviewed by
humans before use. Datasets
should also be selected
carefully to reduce harmful
bias, and AI-generated work
should be clearly disclosed
when appropriate.

## Practical Applications
Generative AI can be used in
education, customer service,
```

crossentropy loss. The model was trained for 10 epochs with a validation split of 20%.

Findings

The model was able to generate new text based on a seed phrase. The generated text followed some patterns from the training data, such as word spacing, punctuation, and sentence-like structure. However, because the model was small and trained for a limited number of epochs, the generated output was not always grammatically perfect.

Training and validation accuracy and loss graphs were used to monitor the learning process. These visualizations helped show whether the model improved during training.

Application Demonstration

A practical content creation example was implemented using a seed phrase such as “once upon a time.” The trained model generated a short creative writing sample. This demonstrates how text generation can support applications such as storytelling, writing assistance, chatbot responses, and content drafting.

Ethical Considerations

Generative AI can create useful content, but it also raises ethical concerns. These include misinformation, plagiarism, biased text, and overreliance on AI-generated material. To reduce these risks, generated content should be reviewed by humans

```
marketing, creative writing,
coding assistance, and digital
media. For example, a company
could use a text generation
model to draft product
descriptions or create chatbot
responses. However, real-world
deployment requires monitoring,
quality control, data privacy
protection, and responsible use
policies.

## Conclusion
This project demonstrated the
basic process of building a
text generation model. The
model was trained on a
public-domain text dataset and
used to generate new text from
seed input. The assignment also
explored GPT architecture,
tokenization, attention
mechanisms, and ethical
considerations. Future
improvements could include
training on a larger dataset,
using word-level or subword
tokenization, increasing model
size, or using a
transformer-based model.
```

before use. Datasets should also be selected carefully to reduce harmful bias, and AI-generated work should be clearly disclosed when appropriate.

Practical Applications

Generative AI can be used in education, customer service, marketing, creative writing, coding assistance, and digital media. For example, a company could use a text generation model to draft product descriptions or create chatbot responses. However, real-world deployment requires monitoring, quality control, data privacy protection, and responsible use policies.

Conclusion

This project demonstrated the basic process of building a text generation model. The model was trained on a public-domain text dataset and used to generate new text from seed input. The assignment also explored GPT architecture, tokenization, attention mechanisms, and ethical considerations. Future improvements could include training on a larger dataset, using word-level or subword tokenization, increasing model size, or using a transformer-based model.